Wireless Performance Analysis

Cetraro Michele Giandomenico Lorenzo
Lombardozzi Christian Minutoli Alessandro

Abstract

This laboratory experience investigates the discrepancies between theoretical maximum goodput and practical application-level performance. Using iperf3 and Wireshark, we measured TCP and UDP goodput across Ethernet-to-Ethernet, Ethernet-to-WiFi, and WiFi-to-WiFi topologies, evaluating 802.11b and 802.11g standards. While the isolated wired baseline closely matches analytical efficiency models, introducing a wireless link severely degrades goodput due to half-duplex CSMA/CA overheads. The data demonstrates that bidirectional contention between data segments and returning ACKs heavily penalizes TCP. Conversely, UDP lacks flow control and suffers severe packet loss when it saturates Access Point buffers. In fully wireless infrastructure setups, the inherent "double-hop" routing penalty and environmental noise drastically amplify contention, which renders nominal physical-layer bitrates poor indicators of actual performance. Ultimately, this comparative analysis highlights how medium access constraints, IP fragmentation, and transport-layer feedback loops strictly dictate real-world wireless efficiency.

1 Introduction

In modern network environments, wireless links introduce unique complexities due to medium access control mechanisms, channel interference, and physical layer overhead. Understanding the practical impact of these wireless links on application-level performance remains a fundamental challenge in network engineering.

The primary objective of this laboratory experience is to critically evaluate the discrepancies between theoretical maximum goodput and practical, real-world measurements. Rather than solely looking at raw physical layer throughput, we focus on the actual rate at which the network successfully delivers useful application data under various structural constraints.

To properly frame our expectations, in the Ethernet-to-Ethernet scenario we deliberately force a bottleneck by constraining the link capacity, as this directly dictates our theoretical predictions; in the remaining topologies, the bottleneck emerges naturally from the wireless link itself, without any artificial limitation. We then establish a theoretical model for maximum protocol efficiency (μ) based on known header overheads and compare these analytical predictions against empirical data. We collect this data across three distinct network topologies—Ethernet-to-Ethernet, Ethernet-to-WiFi, and WiFi-to-WiFi—utilizing both the legacy 802.11b and 802.11g standards. In each scenario, the bottleneck is identified as the slowest link along the path, which serves as the reference for our theoretical predictions. Using `iperf3` for active bandwidth testing and Wireshark for traffic monitoring, we evaluate TCP and UDP performance by systematically varying the interface Maximum Transmission Unit (MTU) and the application payload sizes (L), respectively.

Through this comparative analysis, we systematically investigate how protocol overhead, half-duplex transmission limits, and environmental factors (such as noisy wireless channels) degrade practical goodput compared to our theoretical baseline.

2 Tools and Theory

Goodput G is the rate at which the application receives useful payload L, excluding all protocol overheads, retransmissions, and contention delays. We estimate the theoretical maximum goodput as $G(L) = C \cdot \eta$, where C is the link capacity and η is the protocol efficiency.

For wired Ethernet, the efficiency $\eta_{eth} = \frac{L}{L+H_{tot}}$ is strictly byte-based. With standard protocol headers ($H_{UDP} = 66$ bytes, $H_{TCP} = 78$ bytes including MAC and IP layers) operating over full-duplex links, η_{eth} approaches 97% for large payloads ($L = 1500$). Conversely, Wi-Fi operates over a shared, half-duplex medium governed by CSMA/CA, requiring a time-based efficiency model: $\eta_{wifi} = \frac{T_{payload}}{T_{total}}$, where T_{total} must account for fixed idle times (DIFS, SIFS), random exponential Backoff, and the Layer 2 MAC ACK.

Wi-Fi Degradation Factors & Protocol Impact: Several mechanisms severely penalize T_{total} and reduce η_{wifi} far below 0.5:

- **PHY Generations:** Legacy 802.11b (DSSS) utilizes long PHY preambles and extremely slow basic rates (e.g., 1 Mbps), making even small control frames like MAC ACKs highly time-consuming. 802.11g (OFDM) improves overall efficiency by leveraging shorter preambles and faster basic rates.
- **TCP Bidirectional Contention:** The bidirectional nature of TCP forces data segments and Layer 4 ACKs to alternate on the same half-duplex channel. These ACKs independently contend for the medium, effectively doubling the CSMA/CA time penalty per transaction.
- **UDP Buffer Saturation:** While UDP eliminates transport-layer ACKs and reduces header overhead, its lack of flow control causes severe issues on constrained links. A sender flooding the network at Ethernet speeds will quickly saturate Access Point buffers, resulting in massive packet loss.
- **RTS/CTS Overhead:** Although necessary to mitigate the hidden node problem, the Request/Clear-to-Send handshake introduces significant fixed time delays ($T_{RTS} + T_{CTS} + 2 \times T_{SIFS}$). This overhead heavily penalizes small payloads, rendering the transmission highly inefficient unless large frames are aggregated.

Mixed Scenarios & The Bottleneck Effect: In mixed topologies (e.g., wired server to wireless client), end-to-end goodput is constrained by the slowest link: $G_{e2e} = \min(G_{eth}, G_{wifi})$. Because MAC-layer inefficiencies dictate that $G_{wifi} \ll G_{eth}$, the Access Point (AP) inevitably becomes the bottleneck. When the wired backbone delivers packets faster than the Wi-Fi interface can clear them from the air, the AP's buffers overflow. This leads to packet

drops, forcing TCP to retransmit data, halve its congestion window, and produce highly variable throughput over time.

Experimental Methodology: We use `iperf3` to measure real goodput. For UDP, we dictate payload size via the `-l` flag. For TCP, we actively manipulate the host interface MTU to force specific segment sizes. Finally, Wireshark is used to verify the absence of IP fragmentation and to plot TCP Stream and I/O Graphs, visually confirming the bottleneck-induced throughput fluctuations.

3 Lab Setup and Scenarios

3.1 Testbed Description

We conducted the experiments on a local, controlled testbed composed of two hosts directly connected over the same LAN. Both machines run Ubuntu 25.10 as the operating system. We use `iperf3` version 3.18 to generate traffic and Wireshark version 4.2.2 to capture and analyze packets. We disabled all background applications and system services prior to each experiment to minimize interference from unrelated traffic.

3.2 Experimental Scenarios

We defined three experimental scenarios to progressively increase the complexity of the network path and observe the resulting performance degradation. Across all wireless tests, the network utilizes WPA-PSK/WPA2-PSK (802.11i) security on Channel 1.

3.2.1 ***Scenario 1: Ethernet-to-Ethernet (ETH-ETH).*** In this baseline scenario, both hosts connect via Ethernet to the same switch (Client IP: 192.168.1.3; Server IP: 192.168.1.23; Subnet: /24). Initially, the interfaces negotiate a full-duplex 100 Mbps link to isolate protocol overhead. In subsequent tests, we artificially constrain one interface to 10 Mbps full-duplex to analyze the impact of a strict physical bottleneck on the theoretical predictions.

3.2.2 ***Scenario 2: Ethernet-to-WiFi (ETH-WIFI).*** This mixed topology introduces the wireless link as the natural bottleneck. The server (192.168.1.2) connects via a 100 Mbps wired link, while the client (192.168.1.3) connects wirelessly to the same Access Point (AP). We evaluate the wireless segment under both the 802.11g (54 Mbps) and 802.11b (11 Mbps) standards. Furthermore, we test these configurations in both an isolated baseline environment and an artificially noisy environment.

3.2.3 ***Scenario 3: WiFi-to-WiFi (WIFI-WIFI).*** In this fully wireless scenario, both the client (192.168.1.5) and the server (192.168.1.23) connect to the same AP. Sharing the half-duplex radio medium forces the AP to act as a relay. Since the AP cannot transmit and receive simultaneously, this topology inherently halves the theoretical maximum goodput. We evaluate this setup using both 802.11b and 802.11g standards under three distinct conditions: a baseline environment (2 active stations), a highly congested noisy environment (up to 6 active stations), and a forced RTS/CTS configuration to mitigate hidden node problems.

3.3 Experiment Design

In each scenario, we measure goodput as a function of the payload size L, ranging from small values to the maximum non-fragmented size. We repeat each measurement 10 times to ensure statistical significance, and we report the results as mean, minimum, and maximum across repetitions.

4 Results

4.1 Ethernet-to-Ethernet

Negligible statistical variance (standard deviations ≤ 0.05 Mbps) across all Ethernet-to-Ethernet (ETH-ETH) scenarios confirms a highly stable, collision-free medium, isolating the impact of protocol overhead and bottlenecks.

4.1.1 Scenario 1: 100 Mbps Capacity (TCP). Figure 1 shows asymptotic goodput growth as fixed header overhead amortizes, peaking at 94.07 Mbps ($L = 1500$). The experimental curve consistently trails the theoretical bound, especially at small payloads (36.58 vs. 45.07 Mbps at $L = 64$). This gap arises because processing small payloads requires high packet rates, bottlenecking the CPU and OS before saturating the wire—a limitation exacerbated by TCP Acknowledgment (ACK) generation.

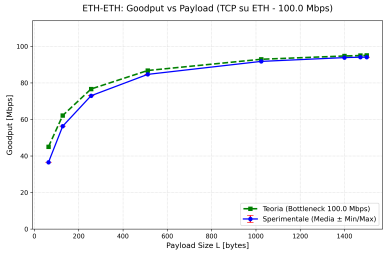


Figure 1: Experimental goodput versus theoretical limit in the ETH-ETH topology (100 Mbps capacity) using TCP.

4.1.2 Scenario 2: 100 Mbps Capacity (UDP). Lacking TCP's ACK overhead, UDP matches theoretical models near-perfectly up to $L = 1024$ bytes (Figure 2). Efficiency peaks at $L = 1472$ bytes (95.48 Mbps), the optimal payload for a 1500-byte MTU. At $L = 1500$ bytes, goodput collapses to 92.39 Mbps. Exceeding the MTU triggers IP fragmentation, splitting the 1528-byte datagram into two physical frames and doubling Layer 2 overhead per payload.

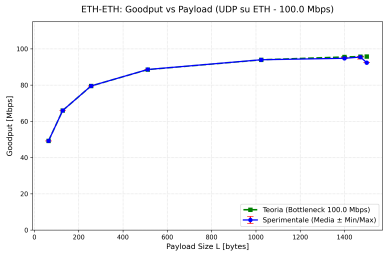


Figure 2: Experimental goodput versus theoretical limit in the ETH-ETH topology (100 Mbps capacity) using UDP. A sharp performance drop occurs at $L = 1500$ bytes due to IP fragmentation.

4.1.3 Scenario 3: 10 Mbps Bottleneck (TCP). Constraining the link highlights flow control limits on a restricted channel (Figure 3). Despite asymptotic growth, experimental goodput remains consistently below theory, especially at small payloads (3.04 vs. 4.51 Mbps at $L = 64$). This gap stems from bidirectional contention: returning ACKs compete directly with data packets for the 10 Mbps bandwidth, preventing the congestion window from fully saturating the forward pipeline.

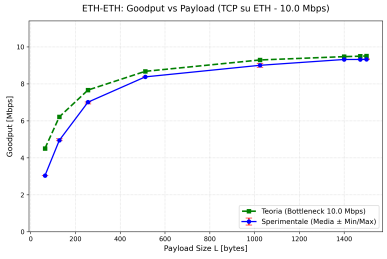


Figure 3: Experimental goodput versus theoretical limit in the ETH-ETH topology using TCP with a constrained 10 Mbps bottleneck.

4.1.4 Scenario 4: 10 Mbps Bottleneck (UDP). Avoiding ACK contention, UDP closely tracks the theoretical model even at small payloads (4.82 vs. 4.92 Mbps at $L = 64$, Figure 5). The slight persistent gap stems from OS buffering delays introduced by traffic shaping.

Wireshark's I/O Graph (Figure 4) visually validates this behavior.

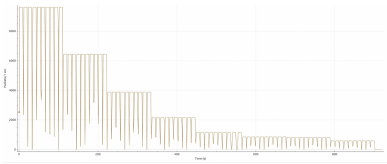


Figure 4: Wireshark I/O Graph (Packets/sec) for the ETH-ETH 10 Mbps UDP scenario. The descending steps correspond to increasing payload sizes, while the high-frequency oscillations reveal the active traffic shaping mechanism.

The descending staircase confirms the inverse relationship between payload size and packet rate. Saturating the link at $L = 64$ requires nearly 10,000 pps, severely bottlenecking the OS, but drops exponentially as L increases. Furthermore, high-frequency intra-step oscillations reveal the traffic shaper actively bursting and throttling datagrams to enforce the 10 Mbps limit.

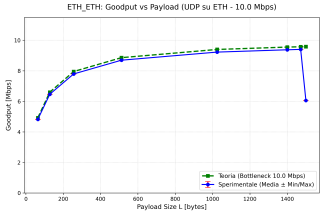


Figure 5: Experimental goodput versus theoretical limit in the ETH-ETH topology using UDP with a constrained 10 Mbps bottleneck. A severe performance drop is visible at $L = 1500$ bytes.

Goodput peaks at $L = 1472$ bytes (9.40 Mbps) before collapsing by 35% to 6.05 Mbps at $L = 1500$ bytes, reiterating the severe fragmentation penalty on constrained links.

4.2 Ethernet-to-WiFi

In this hybrid topology, a wired server connects to a wireless client via an Access Point (AP). While the 100 Mbps wired backhaul is near-ideal, the half-duplex, MAC-heavy wireless hop acts as the strict network bottleneck, severely degrading throughput.

4.2.1 Scenario 5: 802.11b Analysis (11 Mbps). Legacy 802.11b is inherently inefficient due to long PHY preambles. Figure 6 shows TCP goodput plateauing at merely 2.8 Mbps ($L = 1500$), heavily bottlenecked by transmission time overheads.

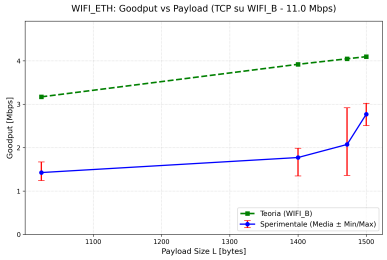


Figure 6: Experimental goodput for 802.11b (11 Mbps capacity) using TCP. Overhead dominates the transmission time for small payloads.

UDP results (Figure 7) follow a similar curve but maintain higher stability. Lacking TCP's bidirectional ACK traffic and congestion window adjustments, the unidirectional UDP flow avoids severe fluctuations. Nonetheless, DSSS physical layer overheads still prevent it from saturating the nominal 11 Mbps limit.

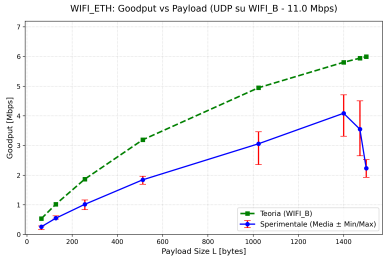


Figure 7: Experimental goodput for 802.11b (11 Mbps capacity) using UDP. The lack of a feedback loop allows for a more consistent unidirectional flow.

4.2.2 Scenario 6: 802.11g Analysis (54 Mbps). Despite a 54 Mbps nominal capacity, 802.11g suffers severe instability under bidirectional TCP load. Goodput peaks at just 7.5 Mbps (Figure 9), with wide standard deviations reflecting frequent MAC-layer retransmissions caused by channel contention.

Wireshark's I/O Graph (Figure 8) diagnoses this degradation, revealing an extreme "stop-and-go" pattern. Prolonged starvation periods (flatlines) are punctuated by violent transmission spikes. This signature confirms an AP bottleneck interacting with TCP congestion control: the 100 Mbps wired server floods the AP, but CSMA/CA gridlock prevents efficient wireless queue clearance. The resulting buffer overflows and packet drops force TCP to collapse its congestion window, periodically probing the network with bursts that instantly re-saturate the AP's memory.

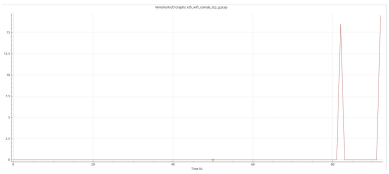


Figure 8: Wireshark I/O Graph for the ETH-WIFI 802.11g TCP scenario. The "stop-and-go" pattern of flatlines and spikes visually confirms buffer overflow at the AP bottleneck and the resulting TCP timeouts.

Conversely, UDP avoids ACK contention, achieving higher raw throughput (Figure 10). However, the sharp goodput collapse at $L = 1500$ corroborates the AP bottleneck: without flow control, the unthrottled UDP stream severely saturates AP buffers, triggering catastrophic packet loss prior to physical radio transmission.

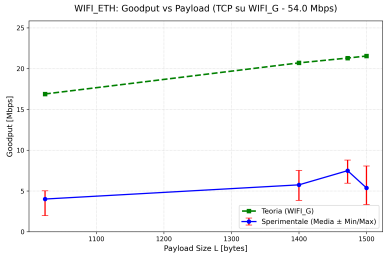


Figure 9: Experimental goodput for 802.11g (54 Mbps capacity) using TCP. Significant variance reflects the instability of the shared wireless medium.

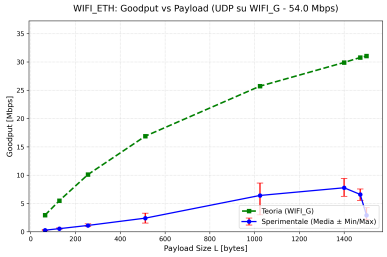


Figure 10: Experimental goodput for 802.11g (54 Mbps capacity) using UDP. A sharp drop at maximum payload size confirms buffer saturation at the AP.

4.3 WiFi-to-WiFi

The WiFi-to-WiFi topology represents the most hostile environment tested. In infrastructure WLANs, the central AP routes all client-to-client traffic, imposing a strict "double-hop" penalty. Transmitting each packet twice over the shared half-duplex medium inherently halves theoretical capacity and drastically amplifies CSMA/CA contention.

4.3.1 TCP Performance and Small Payload Collapse. As Figure 11 demonstrates, the 802.11 MAC layer is extremely fragile under high contention: TCP transfers fail completely for payloads $L \leq 512$ bytes. The unsustainable ratio of control overhead to payload is exacerbated by the half-duplex medium, where receiver ACKs fiercely compete with both incoming server data and AP forwarding transmissions. Even at $L = 1500$ bytes, goodput peaks at a mere 2.88 Mbps (vs. 10.77 Mbps theory). This visually confirms that nominal 54 Mbps physical speeds are irrelevant when CSMA/CA gridlock and double-hop routing choke the MAC layer.

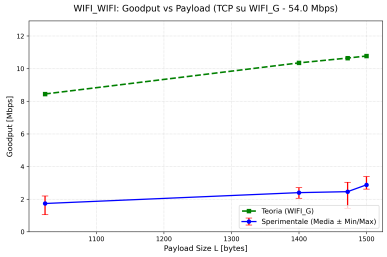


Figure 11: WiFi-to-WiFi TCP Goodput (802.11g, 54 Mbps). The network experiences complete transfer failure for payloads $L \leq 512$ bytes.

4.3.2 UDP Performance and the Fragmentation Cliff. Unlike TCP, UDP lacks transport-layer ACKs, significantly reducing reverse-path contention. As shown in Figure 12, goodput scales positively to a 3.21 Mbps peak at $L = 1400$ bytes. However, the plot highlights a severe fragmentation penalty at $L = 1500$ bytes, where goodput plummets to 2.00 Mbps. Exceeding the 1500-byte MTU forces IP fragmentation, meaning the AP must transmit two physical frames per payload, entirely doubling the already massive PLCP preamble, MAC header, and contention overhead across both wireless hops.

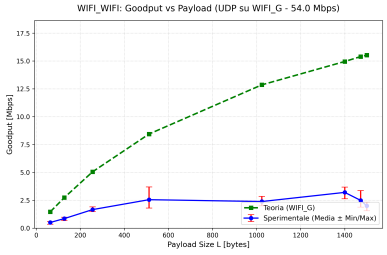


Figure 12: WiFi-to-WiFi UDP Goodput (802.11g, 54 Mbps). Goodput scales positively until a sharp fragmentation penalty occurs at $L = 1500$ bytes.

4.3.3 Impact of Environmental Noise. Repeating the tests under RF interference revealed drastic degradation, nearly halving peak 802.11g TCP goodput (from 2.88 to 1.54 Mbps at $L = 1500$). Environmental noise elevates the physical layer's Bit Error Rate (BER), triggering frame corruption and mandatory MAC-layer retransmissions. TCP inherently misinterprets these physical-layer packet losses and erratic delays as network congestion, prematurely collapsing its congestion window and severely throttling throughput.

5 Conclusions

This laboratory systematically demonstrates the stark contrast between theoretical link capacity and practical application-level goodput. In the stable Ethernet-to-Ethernet baseline, experimental efficiency was near-ideal, disrupted only by the catastrophic throughput penalty of IP fragmentation at $L = 1500$ bytes.

Introducing a wireless hop fundamentally shifts the network bottleneck to the Access Point (AP), proving unequivocally that advertised physical bitrates (e.g., 54 Mbps) are highly misleading metrics. Actual performance is dictated by MAC-layer contention and transport-layer behavior. Under TCP, the half-duplex nature of the AP effectively doubles the required airtime per transaction, as forward data and returning ACKs fiercely compete for the same channel. Conversely, while UDP avoids ACK contention to achieve higher raw throughput, its lack of flow control blindly saturates AP hardware buffers, triggering massive packet loss.

Finally, the fully wireless WiFi-to-WiFi topology represents the most hostile environment. The mandatory "double-hop" AP routing halves theoretical capacity, while amplified CSMA/CA contention triggers complete connection timeouts for small TCP payloads. Compounded by environmental RF interference, these results confirm that practical network performance is strictly bound by medium access overhead, transport-layer feedback loops, and hardware limitations, rendering raw physical transmission speeds largely irrelevant.

6 Appendix

6.1 Iperf3 Script

```
import subprocess
import json
import time
import sys

```

```python
# Configuration
SERVER_IP = "192.168.1.23"
INTERFACE = "wlan0"
PROTOCOL = "TCP"            # "UDP" or "TCP"
CONNECTION_TYPE = "wifi"    # "ethernet" or "wifi"
SCENARIO = "80211g_no_noise_reverse"
LENGTHS = [64, 128, 256, 512, 1024, 1400, 1472, 1500]
REPETITIONS = 10

def set_mtu(iface, mtu):
    try:
        subprocess.run(["ip", "link", "set", "dev", iface, "mtu", str(mtu)], check=True)
        time.sleep(2)
    except subprocess.CalledProcessError:
        print("[!] Error: Cannot change MTU. Are you running as root/sudo?")
        sys.exit(1)

print(f"=== STARTING {PROTOCOL} TEST (REVERSE) ON {CONNECTION_TYPE.upper()} | SCENARIO: {SCENARIO} ===")
results = {str(l): [] for l in LENGTHS}
file_name = f"data_{PROTOCOL.lower()}_{SCENARIO}.json"

for l in LENGTHS:
    print(f"\n[*] Testing Payload L = {l} bytes...")

    if PROTOCOL == "TCP":
        mtu_value = l + 40
        MIN_MTU = 576

        if CONNECTION_TYPE.lower() == "wifi" and mtu_value < MIN_MTU:
            print(f"    [!] SKIPPED: MTU ({mtu_value}) is too low for the Wi-Fi driver.")
            continue

        if mtu_value < MIN_MTU:
            mtu_value = MIN_MTU

        if mtu_value > 1500:
            mtu_value = 1500

        print(f"    Setting MTU to {mtu_value}...")
        set_mtu(INTERFACE, mtu_value)

    for i in range(REPETITIONS):
        print(f"    Run {i+1}/{REPETITIONS} ", end="", flush=True)

        if PROTOCOL == "UDP":
            cmd = ["iperf3", "-c", SERVER_IP, "-u", "-b", "0", "-R", "-l", str(l), "-t", "10", "--json"]
        else:
            cmd = ["iperf3", "-c", SERVER_IP, "-R", "-t", "10", "--json"]

        process = subprocess.run(cmd, capture_output=True, text=True)

        if process.returncode == 0:
            data = json.loads(process.stdout)
            if PROTOCOL == "UDP":
                goodput = data['end']['sum']['bits_per_second'] / 1e6
            else:
                goodput = data['end']['sum_received']['bits_per_second'] / 1e6

            results[str(l)].append(goodput)
            print(f"-> Goodput: {goodput:.2f} Mbps")
        else:
            print("-> [!] Network error. (Ignored)")

        time.sleep(1)

if PROTOCOL == "TCP":
    print("\n[*] Restoring default MTU (1500)...")
    set_mtu(INTERFACE, 1500)

with open(file_name, 'w') as f:
    json.dump(results, f, indent=4)
print(f"\n[V] Test complete! Data saved to '{file_name}'.")
```

Script 1: Python script for iperf3 execution.